Component State & Event Handlers (Part 1c)

1. Destructuring Props & Helper Functions

Helper Functions: Defining functions (like `bornYear`) inside a component is a standard React practice. It keeps the logic scoped precisely to where it is needed.

Destructuring Props: Instead of writing `props.name` everywhere, we extract the values directly in the function parameters. This acts as instant documentation for what the component needs.

```
// ✅ Professional Standard: Destructuring in parameters
const Hello = ({ age, name }) => {

    // Implicit Return (One-liner helper function)
    const bornYear = () => new Date().getFullYear() - age;

    return (
        <div>
            <p>Hello {name}, you are {age} years old</p>
            <p>so you were probably born in {bornYear()}</p>
        </div>
    )
};
```

2. Stateful Components (useState)

Why normal variables fail: If you use `let counter = 0;`, React does not care if the variable changes. The screen will never redraw.

The React Flare Gun: React needs you to shoot a flare into the sky to say *"My data changed! Redraw the screen!"* That flare gun is `useState`.

```
import { useState } from 'react'

const [counter, setCounter] = useState(0);
```

- **Convention:** The first variable is the data (`counter`). The second variable is the exact same word, prefixed with *set* (`setCounter`).
- **Fixed Order:** Index 0 is ALWAYS the value. Index 1 is ALWAYS the remote control function.
- **Use Case:** If data never changes on the screen, don't use state. If data DOES change (like a like button or text input) and the user must see it instantly, use `useState`.

3. Event Handling (The Infinite Loop Trap)

An event handler must be a **function reference**, not a function call. Never execute the handler directly inside the JSX.

❌ BAD (Infinite Loop Crash)

```
<button onClick={increaseByOne()}>
```

The `()` causes the function to run instantly on page load. It updates state, triggers a re-render, calls the function again... resulting in an endless loop.

✅ GOOD (Pass Reference)

```
<button onClick={increaseByOne}>
```

Pass the name of the function. It waits patiently until the user actually clicks.

---

4. React Architecture

Lifting State Up: Data in React is a waterfall—it only flows DOWN. If sibling components (like `<Display />` and `<Button />`) need to share the same data, you cannot put `useState` inside either of them. You must move the state **UP** to their closest common parent (like `App`), and pass the data down via props.

Component Reusability: Create clean, tiny components like `<Button />` and reuse them by passing different props (e.g., passing different `text` and `onClick` handlers to the exact same Button blueprint).

---

5. The Most Important Rule: Changes in state cause re-rendering

Every single time you use a `set...` function, React re-runs your entire component function from top to bottom.

```
import { useState } from "react";

const Display = ({ counter }) => {
  return <div>{counter}</div>
};

const Button = ({ onClick, text }) => {
  return <button onClick={onClick}>{text}</button>
}

const App = () => {
  const [counter, setCounter] = useState(0)
  console.log('rendering with counter value', counter)

  const increaseByOne = () => {
```

```
    console.log('increasing, value before', counter)
    setCounter(counter + 1)
  }

  const decreaseByOne = () => {
    console.log('decreasing, value before', counter)
    setCounter(counter - 1)
  }

  const setToZero = () => {
    console.log('resetting to zero, value before', counter)
    setCounter(0)
  }

  return (
    <div>
      <Display counter={counter} />
      <Button onClick={increaseByOne} text="plus" />
      <Button onClick={setToZero} text="zero" />
      <Button onClick={decreaseByOne} text="minus" />
    </div>
  )
};

export default App;
```

1. **Initial Render:** React calls `App()` for the first time. `counter` is 0. It paints the HTML. React goes to sleep.
2. **State Change:** The user clicks "plus". The handler calls `setCounter(0 + 1)`. The flare gun is fired.
3. **Re-render Magic:** React wakes up, realizes the HTML is outdated, and re-runs the entire `App()` function. It skips the initial 0, sees the counter is 1, hits the console logs, generates brand new HTML, and paints it on the screen instantly.

---

6. Debugging & StrictMode

Why is the console printing twice? React's `<StrictMode>` intentionally renders components twice in development mode. It acts as a safety inspector, stress-testing your code to ensure there are no bugs or unpredictable side-effects.

Seeing two console logs is 100% normal and proves your app is working correctly. It is automatically removed when you deploy your app to production.